FUNCTIONS.md

每个 Java 文件的用途及函数说明，按包层级组织。

目录

1. basic — 基础抽象层
2. aircraft — 飞行器层
3. bullet — 子弹层
4. prop — 道具层
5. strategy — 射击策略层
6. factory — 工厂层
7. manager — 游戏模式管理层
8. dao — 数据持久化层
9. application — 游戏引擎层
10. Activities — 界面层

---

1. basic — 基础抽象层

`AbstractFlyingObject.java`

用途：所有可飞行对象（飞机、子弹、道具）的公共父类，封装坐标、速度、图片、碰撞检测和生存状态。

字段

字段	类型	含义
`WINDOW_WIDTH`	`int`（static）	游戏窗口宽度（像素），由 `surfaceChanged` 动态更新
`WINDOW_HEIGHT`	`int`（static）	游戏窗口高度（像素）
`locationX / locationY`	`int`	对象图片中心的坐标
`speedX / speedY`	`int`	每帧移动像素数（带方向）
`image`	`Bitmap`	对象贴图，懒加载，null 表示未设置
`width / height`	`int`	图片尺寸，-1 表示未设置
`isValid`	`boolean`	生存标记，false 表示已消亡待清除

方法

方法	签名	含义
构造	`AbstractFlyingObject(int x, int y, int sx, int sy)`	设置初始位置和速度

方法	签名	含义
`forward()`	`public void forward()`	按速度移动一帧；横向触碰边界时 `speedX` 反向（弹射效果）
`crash(flyingObject)`	`public boolean crash(AbstractFlyingObject)`	矩形碰撞检测。飞机类 Y 轴碰撞区域缩小为高度的 1/2（驾驶舱判定），子弹/道具为全高
`getLocationX()`	`public int`	返回中心 X 坐标
`getLocationY()`	`public int`	返回中心 Y 坐标
`setLocation(x, y)`	`public void setLocation(double, double)`	直接设置中心坐标（供触摸控制英雄机使用）
`getSpeedY()`	`public int`	返回 Y 轴速度
`getImage()`	`public Bitmap`	懒加载：首次调用时从 `ImageManager` 取图
`getWidth()`	`public int`	懒加载：首次调用时从图片取宽度
`getHeight()`	`public int`	懒加载：首次调用时从图片取高度
`notValid()`	`public boolean`	返回 `!isValid`，用于列表过滤
`vanish()`	`public void vanish()`	将 `isValid` 置为 false，标记对象待销毁

---

2. aircraft — 飞行器层

`AbstractAircraft.java`

用途：所有飞机的抽象父类，在 `AbstractFlyingObject` 基础上增加生命值和射击策略。

字段

字段	类型	含义
`maxHp`	`int`	最大生命值
`hp`	`int`	当前生命值
`direction`	`int`	射击方向（-1 向上，1 向下），默认 0
`power`	`int`	子弹伤害值，默认 0
`strategy`	`ShootStrategy`（private）	当前射击策略，通过 `setStrategy` 注入

方法

方法	签名	含义
构造	`AbstractAircraft(int x, int y, int sx, int sy, int hp)`	初始化坐标、速度和生命值
`decreaseHp(int)`	`public void`	扣除 HP；HP ≤ 0 时自动调用 `vanish()`
`getHp()`	`public int`	返回当前 HP
`setStrategy(ShootStrategy)`	`public void`	运行时替换射击策略（Strategy 模式注入点）
`shoot()`	`public List<BaseBullet>`	委托给当前策略执行射击，返回子弹列表
`getPower()`	`public int`	返回子弹伤害
`getDirection()`	`public int`	返回射击方向

`HeroAircraft.java`

用途：玩家操控的英雄飞机，单例模式，由触摸事件控制位置，初始策略为直射。

模式：Singleton（双重检查锁定，volatile）

字段

字段	类型	含义
`instance`	`HeroAircraft`（volatile static）	单例持有字段

方法

方法	签名	含义
构造（private）	`HeroAircraft(x, y, sx, sy, hp)`	设置 power=30，direction=-1（向上射击），初始策略 DirectShoot
`getInstance()`	`public static HeroAircraft`	获取/创建单例，初始位于屏幕底部中央，HP=1000
`reset()`	`public void`	将 HP、坐标、速度、策略恢复到初始值，用于重新开始游戏
`forward()`	`@Override public void`	**空实现**：英雄机由触摸事件直接 `setLocation`，不走速度移动逻辑
`increaseHp(int)`	`public void`	回复 HP，不超过 maxHp

`AbstractEnemyAircraft.java`

用途：所有敌机的抽象父类，在 `AbstractAircraft` 基础上增加边界消亡逻辑和道具掉落接口。

方法

方法	签名	含义
构造	`AbstractEnemyAircraft(x, y, sx, sy, hp)`	透传给父类
`forward()`	`@Override public void`	调用父类移动后，若 Y 坐标超出下边界则 `vanish()`
`dropProps()`	`public abstract List<AbstractProp>`	子类实现：飞机被摧毁时掉落道具列表

`MobEnemy.java`

用途：普通敌机。直线向下飞行，不射击，不掉落道具，速度快（HP=30）。

方法	含义
构造	设置策略 NoShoot
`shoot()`	返回空列表（不射击）
`dropProps()`	返回空列表（不掉落）

`EliteEnemy.java`

用途：精英敌机。斜向移动、直射、概率掉落道具（HP=60，power=20）。

方法	含义
构造	direction=1（向下射），策略 DirectShoot
`dropProps()`	25% HpProp / 20% FireProp / 25% BombProp / 20% FirePlusProp，其余不掉落

`ElitePlusEnemy.java`

用途：超级精英敌机。斜向移动、散射、概率掉落道具（HP=90，power=20）。炸弹只扣 50 HP，不被一击必杀。

方法	含义
构造	策略 ScatterShoot（3 路散射）
`dropProps()`	与 EliteEnemy 相同概率

`BossEnemy.java`

用途：Boss 敌机。横向来回弹射、环形射击（10 路），不向下移动（HP 由分数波次决定，初始 1000）。不受炸弹影响，击败后得 100 分。

方法	含义
构造	出现在屏幕顶部中央（x=WINDOW_WIDTH/2，y=100），策略 CircularShoot，power=30
`forward()`	调用父类横向弹射逻辑后强制将 `speedY` 归零（保持高度不变）
`dropProps()`	必定掉落一件道具，概率：25% Hp / 20% Fire / 25% FirePlus / 30% Bomb

---

3. bullet — 子弹层

`BaseBullet.java`

用途：子弹抽象父类，持有伤害值，并在越界时自动 vanish。

字段

字段	类型	含义
`power`	`int`	子弹伤害值

方法

方法	含义
构造	初始化坐标、速度、伤害
`forward()`	调用父类移动后检测四边界，任意越界则 `vanish()`
`getPower()`	返回伤害值

`HeroBullet.java`

用途：英雄子弹（向上飞行，Y 速度为负）。`ImageManager` 映射到 `bullet_hero` 贴图。构造透传给 `BaseBullet`，无额外逻辑。

`EnemyBullet.java`

用途：敌机子弹（向下或多方向飞行）。`ImageManager` 映射到 `bullet_enemy` 贴图。构造透传给 `BaseBullet`，无额外逻辑。

---

4. prop — 道具层

`AbstractProp.java`

用途：所有道具的抽象父类，向下飞行，超出边界自动 vanish，被英雄碰到后执行激活效果。

方法

方法	含义
构造	初始化坐标和速度
`forward()`	超出上/下边界时 `vanish()`
`activate(HeroAircraft)`	**抽象**：子类实现具体拾取效果

BombListener.java

用途：观察者接口，由 `BombProp` 持有引用，`Game` 实现该接口并处理爆炸效果（消灭敌机、清除子弹）。

```
public interface BombListener {
    void onBombExplode();
}
```

HpProp.java

用途：血量道具。激活后恢复英雄 30 点 HP（不超过最大值）。

方法	含义
`activate(hero)`	`hero.increaseHp(30)`，然后 `vanish()`

BombProp.java

用途：炸弹道具。激活后通过观察者模式通知 `Game.onBombExplode()` 执行全屏清弹/伤敌效果。

字段

字段	类型	含义
`listener`	`BombListener`	爆炸事件监听器（由 `Game.addPropsWithListener` 注入）

方法

方法	含义
`setBombListener(BombListener)`	注入监听器（观察者注册）
`activate(hero)`	调用 `listener.onBombExplode()`，然后 `vanish()`

FireProp.java

用途：普通火力道具。激活后将英雄射击策略切换为 `ScatterShoot`（3 路散射），并触发 6s 倒计时。

方法	含义
`activate(hero)`	`hero.setStrategy(new ScatterShoot())`，然后 `vanish()`

`FirePlusProp.java`

用途：超级火力道具。激活后将英雄射击策略切换为 `CircularShoot`（10 路环射），并触发/重置 6s 倒计时。

方法	含义
`activate(hero)`	`hero.setStrategy(new CircularShoot())`，然后 `vanish()`

---

5. strategy — 射击策略层

`ShootStrategy.java`（接口）

用途：Strategy 模式的策略接口，将射击行为从飞机类中解耦。

```
public interface ShootStrategy {
    List<BaseBullet> shoot(AbstractAircraft aircraft);
}
```

参数 `aircraft` 提供位置、速度、方向、伤害等上下文信息，策略内部据此生成子弹列表。

`NoShoot.java`

用途：空射击策略，适用于不射击的飞机（MobEnemy）。

方法	含义
`shoot(aircraft)`	返回空列表

`DirectShoot.java`

用途：直射策略，生成 1 颗沿 `direction` 方向飞行的子弹。英雄机生成 `HeroBullet`，敌机生成 `EnemyBullet`。

方法	含义
`shoot(aircraft)`	子弹速度 = aircraft.speedY + direction×10，speedX=0

`ScatterShoot.java`

用途：散射策略，同时生成 3 颗子弹（左/中/右），X 速度分别为 -5/0/+5。

方法	含义
`shoot(aircraft)`	生成 3 颗 speedX = (i-1)×5 的子弹，speedY = aircraft.speedY + direction×8

`CircularShoot.java`

用途：环形射击策略，向 360° 均匀发射 10 颗子弹，速度大小为 20。用于 BossEnemy 及 FirePlusProp 激活后的英雄机。

方法	含义
`shoot(aircraft)`	按 `2π × i / 10` 角度计算每颗子弹的 speedX/speedY

---

6. factory — 工厂层

`EnemyFactory.java`（抽象类）

用途：敌机工厂的抽象基类，持有速度倍率字段供 `Game` 按难度统一设置。

字段

字段	类型	含义
`speedMultiplier`	`double`	速度倍率，默认 1.0

方法

方法	含义
`setSpeedMultiplier(double)`	设置速度倍率（由 `Game.setEnemySpeedMultiplier` 调用）
`createEnemy()`	**抽象**：子类创建并返回具体敌机实例

`MobEnemyFactory.java`

用途：创建普通敌机，随机 X 位置，随机出现在屏幕顶部 5% 区域内，speedY = 10×multiplier，HP=30。

`EliteEnemyFactory.java`

用途：创建精英敌机，speedX = 5×multiplier，speedY = 5×multiplier，HP=60。

`ElitePlusEnemyFactory.java`

用途：创建超级精英敌机，speedX = 8×multiplier，speedY = 7×multiplier，HP=90。

`BossFactory.java`

用途：创建 Boss 敌机，固定出现在屏幕顶部中央（x=WINDOW_WIDTH/2，y=100），speedX=5，speedY=0。

方法

方法	签名	含义
`createEnemy()`	`public AbstractEnemyAircraft`	以默认 HP=1000 创建 Boss

方法	签名	含义
`createEnemy(int hp)`	`public AbstractEnemyAircraft`	以指定 HP 创建 Boss（Hard 模式每波递增 500）

`PropFactory.java`（抽象类）

用途：道具工厂的抽象基类。

```
public abstract AbstractProp createProp(int locationX, int locationY);
```

`HpPropFactory.java`

用途：创建 `HpProp`，speedX=0，speedY=10（向下掉落）。

`FirePropFactory.java`

用途：创建 `FireProp`，speedX=0，speedY=10。

`BombPropFactory.java`

用途：创建 `BombProp`，speedX=0，speedY=10。

`FirePlusPropFactory.java`

用途：创建 `FirePlusProp`，speedX=0，speedY=10。

---

7. manager — 游戏模式管理层

`GameManager.java`（接口）

用途：抽象单机与联机游戏模式的公共操作，供 `Game` 通过依赖注入使用（当前仅实现单机）。

```
public interface GameManager {
    void initialize();   // 初始化（建立连接等）
    void reset();        // 重置游戏状态（保留连接）
    void cleanup();      // 清理资源（关闭连接等）
    boolean isOnline();  // 是否为联机模式
    String getModeName(); // 模式名称字符串
}
```

`SinglePlayerManager.java`

用途：`GameManager` 的单机模式实现，所有网络相关方法均为空实现。

方法	返回值	含义
`initialize()`	void	空实现
`reset()`	void	空实现
`cleanup()`	void	空实现
`isOnline()`	`false`	单机模式
`getModeName()`	`"single"`	模式标识

---

8. dao — 数据持久化层

`GameRecord.java`

用途：游戏记录的数据模型，实现 `Serializable` 以支持对象序列化存储。

字段：`playerName`（玩家名）、`score`（分数）、`timestamp`（时间戳）、`difficulty`（难度字符串）

方法

方法	含义
构造	初始化四个字段
`getPlayerName()`	返回玩家名
`getScore()`	返回分数
`getDifficulty()`	返回难度字符串
`getFormattedTimestamp()`	返回格式化时间字符串（`MM-dd HH:mm`）
`toString()`	返回 `"玩家名, 分数, 难度, 时间"` 格式

`GameRecordDao.java`（接口）

用途：数据访问对象接口，定义增删查操作。

```
public interface GameRecordDao {
    void addRecord(GameRecord record);
    List<GameRecord> getAllRecords();
    void deleteRecord(int index);  // index 为全量排序后的全局下标
}
```

`GameRecordDaoImpl.java`

用途：`GameRecordDao` 的文件实现，使用 `ObjectOutputStream` / `ObjectInputStream` 将 `List<GameRecord>` 序列化到 Android 沙箱路径 `files/records.dat`。

方法

方法	含义
构造	`recordFile = new File(context.getFilesDir(), "records.dat")`
`addRecord(record)`	读取全部记录，追加新记录，整体写回文件
`getAllRecords()`	若文件不存在返回空列表；否则反序列化返回全量列表
`deleteRecord(int)`	读取全部记录，按下标删除，整体写回文件
`saveRecords(List)`	private：使用 ObjectOutputStream 覆盖写入文件

`RankingAdapter.java`

用途：排行榜列表的 `BaseAdapter`，结合 ViewHolder 模式渲染每行记录（排名/玩家名/分数/时间/删除按钮）。

内部接口

```
public interface OnDeleteListener {
    void onDelete(GameRecord record);  // 用户点击删除时回调
}
```

主要方法

方法	含义
构造	接收 `Context`、数据列表、删除回调
`getCount()`	返回记录数
`getItem(int)`	返回指定位置的 `GameRecord`
`getView(int, View, ViewGroup)`	复用 convertView，填充各 TextView 和删除按钮点击事件

---

9. application — 游戏引擎层

`ImageManager.java`

用途：统一管理所有游戏图片的加载与访问。维护一个 `类名→Bitmap` 的映射，使 `AbstractFlyingObject.getImage()` 通过 `obj.getClass().getName()` 自动查找对应贴图。

静态字段（公开 Bitmap）：`BACKGROUND_IMAGE`、`HERO_IMAGE`、`HERO_BULLET_IMAGE`、`ENEMY_BULLET_IMAGE`、`MOB_ENEMY_IMAGE`、`ELITE_ENEMY_IMAGE`、`ELITE_PLUS_ENEMY_IMAGE`、`BOSS_ENEMY_IMAGE`、`PROP_BLOOD_IMAGE`、`PROP_BOMB_IMAGE`、`PROP_BULLET_IMAGE`、`PROP_FIRE_PLUS_IMAGE`

方法

方法	签名	含义
`initImage(Context)`	`public static void`	从 drawable 资源加载全部贴图并建立类名→Bitmap 映射，须在 `surfaceCreated` 时调用
`loadBackground(Context, String)`	`public static void`	根据难度加载背景图（easy→bg，normal→bg2，hard→bg4）
`get(String className)`	`public static Bitmap`	按类名从映射取图
`get(Object obj)`	`public static Bitmap`	按对象运行时类名取图（`AbstractFlyingObject.getImage()` 的实际调用目标）

`MusicManager.java`

用途：统一管理背景音乐（BGM）和音效（SFX）。BGM 用 `MediaPlayer` 处理长音频，SFX 用 `SoundPool` 处理短促音效（最大并发 5 路）。

字段

字段	含义
`isMusicOn`	音效总开关
`bgmPlayer`	普通关卡背景音乐播放器（循环）
`bossBgmPlayer`	Boss 战背景音乐播放器（循环）
`bossIsPlaying`	记录当前播放的是哪首 BGM，暂停后按此恢复
`soundPool`	短音效池（子弹、命中、炸弹、得分、游戏结束）

方法

方法	含义
构造	初始化 SoundPool 和 MediaPlayer
`playBGM()`	播放普通 BGM（未播放时才 start）
`playBossBGM()`	暂停普通 BGM，启动 Boss BGM
`stopBossBGM()`	停止 Boss BGM 并恢复普通 BGM
`resumeBGM()`	暂停后恢复：根据 `bossIsPlaying` 恢复对应 BGM
`pauseBGM()`	同时暂停两个 MediaPlayer

方法	含义
`playShootSound()`	播放射击音效（音量 0.2）
`playHitSound()`	播放命中音效
`playBombSound()`	播放炸弹爆炸音效
`playGameOverSound()`	播放游戏结束音效
`playGetSupplySound()`	播放拾取道具音效
`releaseAll()`	释放所有 MediaPlayer 和 SoundPool 资源（幂等）

`Game.java`

用途：游戏核心，继承 `SurfaceView` 并实现 `SurfaceHolder.Callback`（Surface 生命周期）、`Runnable`（游戏主线程）、`BombListener`（炸弹爆炸）。封装完整的游戏状态、逻辑循环和渲染。

关键字段

字段	含义
`difficulty`	难度字符串（`"easy"/"normal"/"hard"`）
`musicManager`	音乐管理器
`gameManager`	游戏模式管理器（依赖注入）
`onGameEndListener`	游戏结束回调（由 MainActivity 实现）
`mbLoop`	游戏主循环控制标志
`threadLock`	Object 锁，用于保护 gameThread 的启停
`heroAircraft`	英雄机单例
`enemyAircrafts / heroBullets / enemyBullets / props`	游戏对象列表
`score / time`	分数和累计运行时间（ms）
`cycleDuration`	敌机生成/射击周期（ms），Hard 模式动态缩短
`heroShootCycleDuration`	英雄射击周期（ms），Hard 模式动态缩短
`bossScoreThreshold`	触发 Boss 的分数阈值（normal=500，hard=300）
`bossSpawnCount`	已召唤 Boss 的次数，用于 Hard 模式递增 HP
`fireBuffEndTime`	火力 buff 到期时间戳（ms），-1 表示未激活
`FIRE_BUFF_DURATION`	火力 buff 持续时长，固定 6000ms

内部接口

```
public interface OnGameEndListener {
    void onGameEnd(int score);  // 游戏结束时，由 Game 调用，MainActivity 实现跳转
}
```

Setter / Getter

方法	含义
`setGameManager(GameManager)`	注入游戏模式管理器
`setOnGameEndListener(OnGameEndListener)`	注入游戏结束回调
`setGameMode(String)`	设置游戏模式（"single"/"multi"）
`setDifficulty(String)`	设置难度
`setSoundEnabled(boolean)`	设置音效开关

SurfaceHolder.Callback 生命周期

方法	含义
`surfaceCreated(SurfaceHolder)`	等待旧线程停止 → 重置英雄机 → 应用难度参数 → 启动游戏线程 → 播放 BGM
`surfaceChanged(SurfaceHolder, int, int, int)`	更新 `WINDOW_WIDTH/HEIGHT`，按屏幕尺寸缩放背景图
`surfaceDestroyed(SurfaceHolder)`	停止主循环，释放音乐资源

主线程 `run()`

每 16ms 循环一次：`action()` → 锁定 Canvas 调用 `draw(canvas)` → `Thread.sleep(16)` → 解锁提交画布。

核心逻辑方法

方法	含义
`action()`	主逻辑帧：更新时间 → Hard 模式动态调参 → 敌机生成 → 子弹/飞机/道具移动 → 碰撞检测 → 清理失效对象 → 游戏结束判断
`timeCountAndNewCycleJudge()`	private：累加 cycleTime，达到 cycleDuration 时返回 true 并重置（触发一轮生成+射击）
`shootAction()`	private：遍历所有敌机，调用 `shoot()` 将子弹加入 `enemyBullets`
`heroShootAction()`	private：计时达到 heroShootCycleDuration 时触发英雄射击并播放射击音效
`bulletsMoveAction()`	private：调用所有子弹的 `forward()`
`aircraftsMoveAction()`	private：调用所有敌机的 `forward()`

方法	含义
`propsMoveAction()`	private：调用所有道具的 `forward()`
`crashCheckAction()`	private：① 敌弹打英雄机；② 英雄弹打敌机（含得分、道具掉落、Boss 触发、BGM 切换）；③ 英雄机与敌机直接碰撞；④ 英雄机碰道具（激活效果、火力 buff 计时）
`postProcessAction()`	private：用 `removeIf(notValid)` 清除四个列表中的失效对象
`addPropsWithListener(List)`	private：将新道具加入 `props` 列表，并为 BombProp 注册 `this` 作为 BombListener
`onBombExplode()`	BombListener 实现：清除所有敌弹；消灭/削减各类敌机（BossEnemy 免疫）；播放炸弹音效
`setEnemySpeedMultiplier(double)`	private：对三个普通敌机工厂统一设置速度倍率
`applyDifficultySettings()`	private：按当前难度初始化 bossScoreThreshold、enemyMaxNumber、cycleDuration、heroShootCycleDuration、speedMultiplier、背景图

绘制方法

方法	含义
`draw(Canvas)`	绘制滚动背景 → 敌弹 → 英雄弹 → 道具 → 敌机 → 英雄机 → HUD
`paintImageWithPositionRevised(Canvas, List)`	private：将列表中各对象以中心坐标为基准绘制到 Canvas
`paintScoreAndLife(Canvas)`	private：绘制 SCORE、LIFE、难度标签，若火力 buff 激活则显示倒计时

生命周期控制

方法	含义
`pause()`	停止主循环（mbLoop=false），暂停 BGM
`resume()`	若线程已停止则重新启动游戏线程和 BGM
`reset()`	在 threadLock 内重置全部游戏状态（分数、列表、Boss 状态、火力 buff、英雄机），重新应用难度参数并恢复 BGM
`cleanup()`	在 threadLock 内停止线程（最多等待 3s）、清空列表、释放全部音乐资源

---

10. Activities — 界面层

`StartActivity.java`

用途：游戏启动页，提供单机/联机模式选择按钮和音效开关，使用 `SharedPreferences` 持久化音效设置。

主要逻辑

方法	含义
`onCreate`	初始化控件，读取音效状态，注册按钮点击事件
`onResume`	返回时重新读取音效状态同步 Switch 显示
单机按钮点击	携带 `mode="single"` 和 `soundEnabled` 跳转 `DifficultyActivity`
联机按钮点击	携带 `mode="multi"` 和 `soundEnabled` 跳转 `DifficultyActivity`

`DifficultyActivity.java`

用途：难度选择页，接收 mode 和 soundEnabled，提供 Easy/Normal/Hard 三档选择后跳转 `MainActivity`。

主要逻辑

方法	含义
`onCreate`	展示当前模式名称，绑定三个难度按钮和返回按钮
`launchGame(mode, sound, difficulty)`	private：携带三个参数 Intent 跳转 `MainActivity`

`MainActivity.java`

用途：游戏主界面容器。创建并托管 `Game` SurfaceView，处理暂停对话框，响应游戏结束回调后跳转 `EndActivity`，并根据 `EndActivity` 返回结果决定重新开始/返回主菜单/退出。

实现接口：`Game.OnGameEndListener`

主要逻辑

方法	含义
`onCreate`	读取参数 → 创建 `SinglePlayerManager` → 创建 `Game` 视图并注入管理器/难度/监听器 → 加入 FrameLayout → 绑定暂停按钮
`showPauseDialog()`	private：调用 `game.pause()` 后弹出 AlertDialog，选项：继续/重新开始/回到主界面/退出
`onGameEnd(int score)`	`OnGameEndListener` 实现：跳转 `EndActivity` 并传入分数（startActivityForResult）
`onActivityResult`	根据 `EndActivity` 的返回码（RESTART/MENU/EXIT）执行对应操作
`navigateToStart()`	private：清空回退栈跳转 `StartActivity`
`onDestroy`	调用 `gameView.cleanup()` 释放资源

`EndActivity.java`

用途：游戏结束页，显示最终分数，提供查看排行榜、重新开始、返回主菜单、退出四个选项，通过 `setResult` 将用户选择返回给 `MainActivity`。

返回码常量

常量	值	含义
`RESULT_RESTART`	100	重新开始
`RESULT_MENU`	101	返回主菜单
`RESULT_EXIT`	102	退出游戏

主要逻辑

方法	含义
`onCreate`	读取 Intent 中的 `score`，显示分数，绑定四个按钮
`onBackPressed`	按返回键视为"返回主菜单"（setResult RESULT_MENU）

`RankingActivity.java`

用途：排行榜页，按难度分 Tab（Easy/Normal/Hard）展示历史记录，支持点击删除单条记录，数据通过 `GameRecordDaoImpl` 从本地文件读取。

字段

字段	含义
`allRecords`	全量记录（全难度，按分数降序排列）
`shownRecords`	当前 Tab 过滤后的显示数据
`currentDifficulty`	当前选中 Tab 的难度字符串

主要方法

方法	含义
`onCreate`	初始化 DAO、ListView、Tab 按钮，默认显示 Easy Tab
`loadAllRecords()`	private：从 DAO 读取全量记录并按分数降序排序
`switchTab(String)`	private：更新 Tab 高亮色，调用 `refreshShownRecords()`
`refreshShownRecords()`	private：清空 shownRecords，从 allRecords 过滤当前难度，通知 Adapter 刷新
删除回调	在 allRecords 中找到全局下标，调用 `dao.deleteRecord(index)` 后刷新